

Autonomous Software Agents Project Report

Daniele Buondonno

267888

daniele.buondonno@studenti.unitn.it

Sasha Petkovic

264689

sasha.petkovic@studenti.unitn.it

1 Introduction

In this report, we will give an overview of our project for the Autonomous Software Agents course. The project's goal is to create and develop autonomous agents that can play the game Deliveroo.js.

The game consists in picking up and delivering parcels, which yield different reward values. The game's environment is competitive, as rival agents can be present. The project is divided into two parts.

- **BDI Agent:** An autonomous agent that uses the Belief-Desire-Intention architecture to observe and perform actions in an environment. This architecture allows the agent to maintain a set of beliefs about the environment obtained through sensing. Through the belief set, the architecture allows the agent to compute possible options (desires) along a utility score for them. The option with the best utility score is selected and performed, if possible.
The architecture prescribes that a loop is present, where options continue to be generated given new observations, and where the current Intention can be revised.
- **LLM Agent:** An agent that acts as a partner to the BDI agent. It performs like the BDI agent through a setup composed of API calls and prompts. It must be able to understand the rules of the game and the environment around it.
It must also be able to understand, evaluate and perform missions delivered to it through the game chat, which can yield positive or negative results.

We will first highlight the structure of the BDI agent, explaining its different components and operations. We will go through the data structures used, the strategies we decided to implement and our reasoning behind both. We will then explain the LLM's agent structure and operations. To conclude we will give some brief results and our thoughts for further improving our solutions.

2 Belief

In the Belief-Desire-Intention architecture, an agent's belief set represents what it believes to be true about the environment. The term belief is not used casually: what the set contains is not ground truth, but what the agent has deduced from present and past observations, which may no longer hold. As we will see, in our solution we try to keep the most up to date informations and forget what we confirm to be outdated, so priority is given to the information we're most confident about.

We will now go over the different classes implementing the agent's Belief Set. Every class is instantiated per agent rather than shared at module level, which is what allows the BDI and the LLM agent to run in the same process.

2.1 Me and Agents

The **Me** class holds the agent's identity, its position and its score. For the agents' positions, we must consider when gating or considering fractional positions, as they indicate that an agent is attempting to move or is in the middle of one. For the agent itself, we gate fractional positions, avoiding inconsistent information about our own position and making wrong assumptions about a move through the environment.

The class **Agents** holds the latest observed state of every other agent, indexed by id. For this class we save fractional positions because they tell us that two tiles are occupied rather than one. In fact, a stationary agent occupies only one tile, while a moving agent occupies both its current tile and the tile it's attempting to move towards.

2.2 Parcels

This class represents what the agent believes about the parcels it carries or has observed, using three maps indexed by parcel id.

In **Visible** we store all the parcels the agents saw during the latest sensing and it is rebuilt every time the agent performs sensing. These observed parcels are the ones the agent is most confident about, as it can currently see them.

Known contains free parcels observed in the past, together with the time of their last observation. This timestamp is used to compute estimate parcel reward when decay is active.

To maintain this data structure as accurate as possible, whenever a known parcel's location is observed, and the parcel is not present, it is removed and forgotten by the agent. If the agent has a partner, this map also holds the information of the parcels reported by the partner. A report does not overwrite an already known parcel.

Carried holds the parcels the agent's currently carrying. It is updated through sensing and through confirmed pickup and putdown actions.

2.3 Crates

This class is specifically designed to represent the agent's beliefs about the crates in the map, if present. A **Known** map is also used here, it works the same way as for the parcels, only this time it stores crates. We also made use of a **byPosition** map, indexed by a coordinate string. **Known** remains the main belief representation, while the positional index allows to avoid repeatedly scanning all known crates during planning.

2.4 Partner

This class stores the latest information received from the teammate and manages the data exchanged between the two agents. The shared state includes position, carried parcel ids, carried count and carried reward. The agents also exchange directly observed free parcels and announce the parcel they are currently pursuing. Before committing to a pickup, an agent compares its distance to the parcel with the distance announced by the partner and yields when the partner is closer. Ties are resolved through a deterministic ordering of the agent ids. Their communication is overhead free as they only send new information to each other, avoiding duplicate information.

2.5 World

This class stores the map, the current configuration, the positions covered by the latest sensing event, and the known spawner and delivery tiles.

An important derived quantity is **decayPerMove**, which expresses reward decay in terms of movement cost.

$$\text{decayPerMove} = \begin{cases} \frac{\text{movementDuration}}{\text{decayInterval}} & \text{if } \text{decayInterval} > 0 \\ 0 & \text{otherwise} \end{cases}$$

This conversion allows pickup and delivery utilities to estimate reward loss directly from shortest path length. The movement duration is also used separately to express how many movements have passed since a spawner was last observed.

Each spawner and delivery tile stores the time it was last observed and whether it belongs to a topology that supports a repeatable pickup-delivery cycle. This distinction matters on maps with one way regions: a tile may be reachable while leaving the agent unable to return to a useful spawn or delivery area. These delivery tiles are preferred during deliberation, while other reachable delivery tiles remain available as a fallback. Exploration is more conservative and considers only spawners from which an operational delivery can still be reached.

3 Action Based Belief Revision

Beliefs are revised not only through sensing, but also through the confirmed outcome of actions. Waiting for a later observation would leave the next planning cycle using a state that is already known to be outdated. After a successful movement, the integer position is recorded immediately. Pickup and putdown outcomes revise the visible, known and carried parcel maps, while a successful crate push updates both crate indexes. These revisions are applied only after the action reports success. Sensing events remain authoritative and can correct the local state.

4 Evidence Based Belief Revision

The belief state preserves the latest information, it does not make use of histories or probabilistic values. Other agents are represented only by the latest sensing, no future trajectories are predicted. Parcels and crates are treated differently, as their last known positions remain useful outside the current field of view. They are retained until direct observation contradicts the belief, an action outcome revises it, or, for parcels, the estimated reward becomes negative. This is an intentional tradeoff. A predictive model could produce an incorrect extrapolation that can suppress valid intentions.

5 Desires and Intention Loop

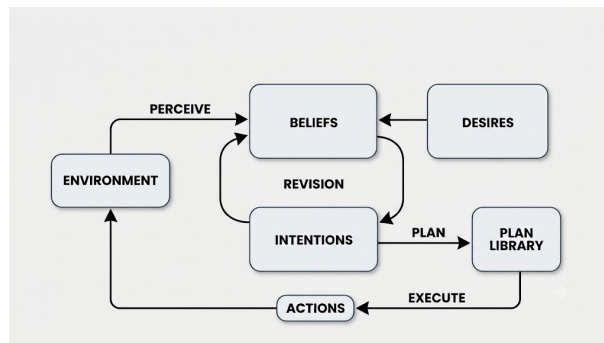


Figure 1: The BDI Loop.

During every BDI iteration, the agent generates desires that are currently achievable from its beliefs. Each desire contains a type, a target, an estimated utility and the identifiers required for planning.

The agent supports three autonomous desire types:

- `go_pick_up`: it defines the desire of picking up a parcel.
- `go_deliver`: it defines the desire of delivering the currently carried parcels.
- `go_to_spawner`: this type of desire is used as an exploration option. It is designed to be generated only when the agent currently doesn't have the option of either delivering or picking up parcels.

The LLM layer may also introduce temporary external objectives. They enter the same deliberation loop as standard desires, while their interpretation and creation are described in the LLM section.

5.1 Desires Generation

Utility estimation uses shortest path distances through a BFS search. A BFS from the current position provides the distance to every reachable pickup, delivery and exploration candidate. For each possible pickup tile, another search evaluates following routes towards available delivery tiles. This lets the agent estimate the complete cost of collecting and later delivering the parcels. Let C be the set of currently carried parcels, b a batch of parcels located on the same tile, and δ the estimated reward decay per movement. For a set of parcels S delivered after D movements, we define first the estimated reward for delivering a set of parcels after a given amount of movements.

$$R(S, D) = \sum_{p \in S} \max(0, r_p - \delta D) \cdot m_{parcel}(r_p)$$

Where r_p is the current estimated reward for p . The parcel multiplier is equal to one when no strategy is active.

5.1.1 Pickup Utility

A pickup action collects every parcel on the current tile, so parcels sharing the same coordinates are evaluated as a batch. A batch is eligible only when:

- its tile is reachable.
- the amount of parcels it contains is allowed by the carry stack strategy.
- the agent's partner hasn't claimed it.
- at least one reachable delivery tile from the batch's position produces a positive expected utility.

For batch b and delivery tile d , the total distance is:

$$D(b, d) = d(agent, b) + d(b, d)$$

The pickup utility is:

$$U_{pickup}(b, d) = \frac{[R(b, D) + R(C, D)] \cdot m_{stack}(|C| + |b|) \cdot m_{delivery}(d)}{\max(1, D)}$$

The reward of the carried parcels is included because they also decay while the agent reaches the batch and then the delivery tile. Since all reachable delivery candidates are evaluated, the selected delivery is not necessarily the closest one. For each eligible batch, the delivery tile producing the highest positive utility is chosen, and the resulting pickup desire is added to the desire set.

5.1.2 Delivery Utility

This desire can be generated if the agent is carrying at least 1 parcel and the current stack strategy allows delivery. For a delivery tile d at distance D , its utility is:

$$U_{delivery}(d) = \frac{R(C, D) \cdot m_{stack}(|C|) \cdot m_{delivery}(d)}{\max(1, D)}$$

Only delivery candidates with positive expected utility are retained. Operational delivery tiles are prioritised because they allow the agent to continue a repeatable pickup-delivery cycle. When no operational delivery is available, all reachable deliveries act as a fallback.

5.1.3 Exploration Utility

This desire is generated only when no valid pickup and delivery desires can be generated. For this utility we only consider spawners that are reachable, outside of the current sensing area and connected to an operational delivery tile.

Their utility is computed with the following formula.

$$U_{exploration}(s) = \frac{movesSinceCheck(s)}{\max(1, d(agent, s))}$$

This formula favours spawners that have not been observed recently while still paying mind to their distance. Exploration may also be generated when the agent carries parcels but no valid delivery is currently available, preventing it from remaining indefinitely idle.

5.2 Intention Loop

The intention loop continuously alternates deliberation, planning, execution and belief revision. During each iteration, the agent:

1. generates the desires that remain valid under the latest beliefs and active strategies.
2. filters desires that are temporarily obstructed or currently unplannable.
3. revises or selects the current intention.
4. plans and executes the intention's action.
5. reconciles its outcome with the beliefs, planning state and any external objective.

5.3 Intention Revision

Intention revision balances commitment with reactivity. The current intention is matched against newer desires through an identifier. If no matching valid desire remains, a new intention is selected. Otherwise, the following rules are applied:

- **External objectives:** an active objective introduced by the LLM has priority over autonomous desires and remains selected while it is valid.

- **Crate-task commitment:** a delivery using an active crate handling task is preserved until the task, intention or crate configuration changes.
- **Immediate pickup:** a valid batch on the current tile can be selected because picking it up requires no additional costs.
- **Exploration persistence:** an exploration intention is maintained while its corresponding desire remains valid.
- **Utility improvement:** an intention replaces the current intention only when its utility is strictly greater.
- **Delivery protection:** an active delivery intention can be replaced only by a pickup that has both greater utility and shorter distance than the current delivery.

If none of these replacement conditions is satisfied, the current intention is maintained. This policy prevents unnecessary switching, as it reacts to invalid goals, immediate opportunities and more convenient alternatives.

6 Planning

Once an intention is established by the agent, the next step is planning its execution. A plan is a sequence of steps that concludes with the fulfillment of the plan's objective.

To establish a plan, we have different options. The first is the more traditional solution for artificial intelligence planning found in the PDDL language. Given a domain, with a set of rules, states and possible actions for all the states, it allows to search and find a plan that satisfies a given task, if it's possible.

The second option is less traditional and more standard programming oriented, as we simply make use of a search algorithm to find a path to a desire's target and execute its action.

We decided to use PDDL only when strictly needed, while we 'defaulted' to using a BFS search for all other cases. We will explain in the following sections our motivations and use of them.

6.1 BFS Search and Planning

For all the cases where PDDL is not strictly needed we opted to use our version of a BFS search. The Breadth First Search (BFS) is a classic search algorithm that can be used with graphs or grid like environments. It performs a search by expanding all nodes in the current depth. Once every node in the current depth has been explored, the search proceeds to the next depth. This process continues until the goal is found. As mentioned earlier, this search is optimal for domains where the costs for all actions are equal. This allowed us to use our BFS algorithm to compute shortest path distances for better utility value computations.

Once an intention is established, the agent uses the BFS search to find the shortest path to the intention's target. Once the path is found, the agent follows it and, once it has reached the target destination, performs the action.

Using this search, the BDI agent ended up being very fast and efficient. This is not the case for PDDL planning. Also, we note that this solution is much more flexible to intention revision, or changes in the environment that require a different path to be taken to fulfill the current intention. This is due to the fact that computing another BFS search is much faster than going through the PDDL solver.

6.2 PDDL

We used PDDL for planning only in cases where it was strictly needed, as its performance is greatly worse than the one we implemented through a simple search.

The slower performance, especially in our case, is given by the fact that to find a plan, PDDL goes through its own solver. This process is slower than simply performing a search.

We decided to use PDDL for maps with crates. Specifically, when an agent encounters a crate in its path it calls the PDDL solver to find a path that allows the agent to move towards its current intention's target. In these cases, the agent stops for some seconds, waiting for the solver to receive the problem's details and for it to send back the satisfying plan. Given that BFS alone couldn't solve this kind of problem, we find that relegating PDDL for this use case alone is a satisfying solution, as the agent performs very efficiently in the majority of game worlds.

We note that PDDL is also less flexible to changes in the environment or intention revision. If a path needs to be redefined due to changes in the environment, such as new obstacles, or because of a change of the current intention, the agent has to sit and wait for the whole process again.

7 LLM Agent and Communication

7.1 LLM Agent Architecture

The LLM Agent operates as a complementary, reasoning partner alongside the BDI agent. Rather than relying solely on hardcoded utility formulas and planning, the LLM agent leverages OpenAI's API to interpret game states, evaluate natural-language events, and decide optimal actions. It is also able to interpret, evaluate and perform missions given to it through the in game chat. It can also communicate with the BDI agent. The LLM Agent's architecture performs these operations in order to function.

1. **State Serialization and Prompting:** On each sensing tick the agent serializes its localized belief state, including grid coordinates, carried parcel rewards, visible decay rates, and map obstacles into a structured prompt context.
2. **Action Reasoning:** The system prompt embeds Deliveroo.js game rules, movement constraints, and action formats. The LLM outputs structured responses, indicating its own reasoning and computations, if needed. It then performs the required actions to fulfill its own intentions.
3. **Mission Evaluation:** As mentioned earlier, missions can be assigned from the server via game chat. They present side objectives with variable positive or negative rewards. The LLM agent parses these natural language announcements, evaluates the risk versus reward ratio based on its and the game's current status, and decides whether to accept, delegate, or ignore the mission.
4. **Robustness and Fallback:** To handle API latency or invalid LLM generations, the agent validates output paths using the BFS/PDDL planner before execution, falling back to basic deterministic actions if a timeout occurs.

7.2 Multi-Agent Communication Protocol

The BDI agent and the LLM Agent form a team. They communicate with each other, adding information to each other's belief and share other informations.

The LLM agent can let the BDI agent know about some ongoing mission, active multipliers or potential negative rewards for specific actions. They also let each other know about their

intentions, to avoid edge cases where both pursue the same objective.

Some missions may require strict cooperation between the two agents. The LLM agent is able to send simple instructions to the BDI agent, through either target coordinates or simple instructions. We also implemented a simple functionality that lets them choose which of the two should perform an intention and pick the one that is in the better position to efficiently perform it.

To maximize overall team utility and prevent redundant efforts, the BDI and LLM agents communicate asynchronously using the game server’s socket connection.

8 Challenge Benchmarks and Conclusions

To give an evaluation of the performance of our agents, we decided to measure the BDI agent’s performance by trying to replicate the environment found during the first challenge. The first challenge only involves the BDI agent. The agent has to simply collect as many points as possible in 5 minutes while competing with other agents.

We ran three maps, a standard map with only the ‘classic’ tiles, a map with unidirectional tiles, and a map with crates. For the first map, we paired it against 8 intelligent NPCs, with 15 maximum parcels, for the second, we paired it against 3 intelligent NPCs with 8 maximum parcels. For the third map, we decided to maintain the standard settings for the map, given the small space.

Table 1: BDI Only Performance

Level	Score
26c1_1	2097
26c1_4	1804
crates_one_way	1273

For the performance of the LLM agent, the lack of a standard benchmark made us choose not to report the results of a benchmark made up by us. We obviously tested both the functionality and performance of the LLM agent, both alone and while cooperating with the BDI agent, and we found the results satisfying. The LLM agent correctly understands the rules of the game, the missions given to it and it correctly performs actions in the environment. If its performance is a concern, we left instructions in our GitHub repo, and we apologise if we missed the presence of a standard benchmark to utilise.

We believe that the project’s requests were satisfied, and that the overall performance can be considered quite good.